

Poly Quiz
Protocole de communication

Version 1.2

Historique de version

Date	Version	Description	Auteur
2024-09-23	1.0	Version soumise à la revue par l'équipe	Maxime Pageot
2024-09-26	1.1	Version après feedback	Maxime Pageot
2024-09-26	1.2	Version révisée	Hamza Boukaftane

Table des matières

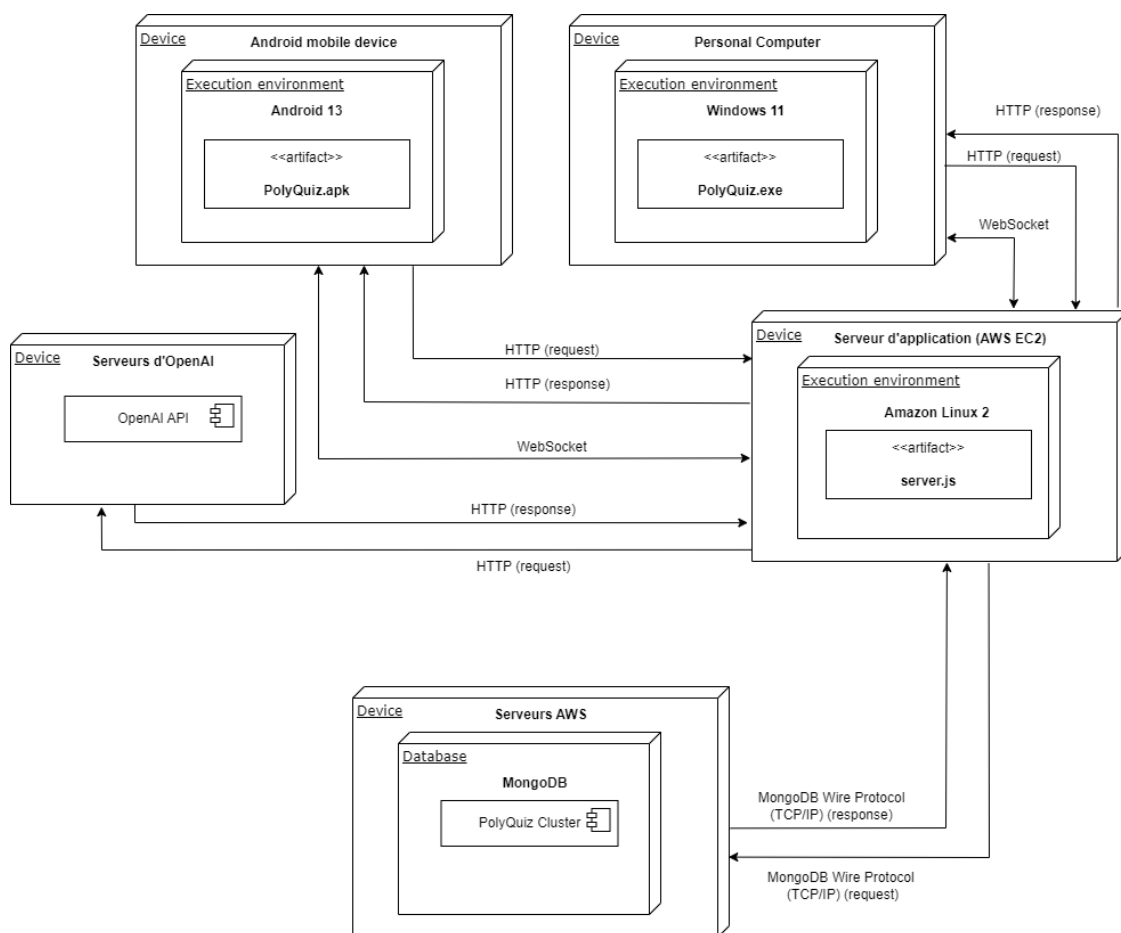
1. Introduction	4
2. Communication Client-Serveur	4
3. Description des requêtes	5
3.1 Protocole HTTP:	5
3.1.1 Définition des paramètres URL:	5
3.1.2 Banque de Quiz:	5
3.1.2 Actions de gestion de compte:	6
3.1.3 Informations sur l'état du compte	7
3.1.4 Actions d'amitié et blocage	8
3.1.5 Actions incluant une dépense:	9
3.2 Protocole WebSocket:	11
4. Interfaces et Enums utilisés	17
Interfaces Communes:	17
Enum Communs:	19
Interfaces client:	20
Interfaces Serveur:	20

Protocole de communication

1. Introduction

Ce document a pour but d'informer les clients et les développeurs des protocoles de communication HTTP et Socket utilisés par l'application Poly Quiz. La deuxième partie présente le schéma de déploiement et les voies de communications. La troisième partie décrit les requêtes HTTP disponibles via l'api et la forme des réponses. La quatrième partie liste les messages envoyés par les sockets web.

2. Communication Client-Serveur



Les clients communiquent par HTTP et par socket avec le serveur d'application, qui interagit avec notre instance mongoDB sur un autre serveur AWS par le protocole MongoDB Wire et avec l'API (application programming interface) de OpenAI par HTTP.

3. Description des requêtes

3.1 Protocole HTTP:

3.1.1 Définition des paramètres URL:

Paramètre d'URL	Description
:quizId	id du Quiz sélectionné
:title	titre du quiz qu'on veut vérifier qu'il n'est pas utilisé
:username	le nom du compte concerné par la requête
:userId	l'id du compte concerné par la requête
:emitterId	id du compte dont l'action initie une relation entre deux utilisateurs (compte qui envoie une demande d'ami, compte qui en bloque un autre)
:targetId	id du compte qui est le receveur de la relation entre deux utilisateurs (reçoit la demande d'ami, est bloqué)
:itemId	id de l'item acheté
:gameId	id de la partie payante qui doit être testée

3.1.2 Requêtes - Banque de quiz:

Les requêtes envoyées au serveur de l'application permettent de créer, modifier, effacer ou encore rechercher de l'information sur les quiz.

Chemin (URL)	Méthode	Requête (corps)	Réponse	
			Code	Body
/quiz/	GET	-	200	Quiz[]
/quiz/visible	GET	-	200	Quiz[]
/quiz/:id	GET	-	200	Quiz[]

			404	"Quiz not found"
/quiz/	POST	{ Quiz }	201	-
/quiz/	PUT	{ Quiz }	200	-
/quiz/:quizId	PATCH	{ visible : boolean }	200	-
			401	"unauthorised"
			404	"Quiz not found"
/quiz/:QuizId	DELETE	-	204	-
			401	"unauthorised"
			404	"Quiz not found"
/quiz/checkTitleUniqueness/:title	POST		200	boolean

3.1.2 Requêtes - Actions de gestion de compte:

Les requêtes envoyées au serveur de l'application permettent de créer, modifier, effacer ou encore rechercher de l'information sur les comptes utilisateurs de Poly Quiz.

Chemin (URL)	Méthode	Requête (corps)	Réponse	
			Code	Body
/auth/admin-password	POST		200	authorizationNotice
			401	"incorrect admin password"
/account/login/:username	POST		201	authorizationNotice
			401	"unauthorized: name or password invalid"
/account/	DELETE		204	-

login/:username				
			401	“unauthorized: name or password invalid”
/account/login	POST		200	authorizationNotice
			401	“password invalid”
			404	“no user with that username found”
/account/:userid/image	PATCH	ImageData	200	“profile picture changed”
			401	“authorization token invalid”
			415	“cannot read image”
/account/:userid/config	GET	-	200	Config
			404	“account not found”
/account/:userid/config	PATCH	{Configuration:Config}	200	Config
			404	“account not found”

3.1.3 Requêtes - Informations sur l'état du compte

Les requêtes envoyées au serveur de l'application permettent de rechercher de l'information sur l'historique d'action, l'argent, l'expérience et le prestige des comptes utilisateurs de Poly Quiz.

Chemin (URL)	Méthode	Requête (corps)	Réponse	
			Code	Body
	GET	-	200	String[]

/account/:userid/actionHistory			404	"account not found"
/account/:userid/money	GET	-	200	number
			404	"account not found"
/account/:userid/experience	GET	-	200	Experience
			404	"account not found"
/account/:userid/Challenges	GET	-	200	Challenge
			404	"account not found"

3.1.4 Requêtes - Actions d'amitié et blocage

Les requêtes envoyées au serveur de l'application permettent de gérer les amitiés entre les différents comptes utilisateurs de Poly Quiz.

Chemin (URL)	Méthode	Requête (corps)	Réponse	
			Code	Body
/friends	POST	-	204	-
			404	"account not found"
			401	"invalid authorization token"
/friends/:emitterId/:targetId	DELETE	-	204	-
			404	"account not found"/ "friend relation does not exist"
			401	"invalid authorization token"

/friends/accept/:emitterId/:targetId	POST	-	204	-
			404	"account not found"/ "friend relation does not exist"
			401	"invalid authorization token"
/block/:targetId	GET	-	200	userInfo[]
			404	"user not found"
			401	"invalid authorization token"
/block/:emitterId/:targetId	GET	-	200	boolean
			404	"user not found"
			401	"invalid authorization token"
/block/emitterId/:targetId	POST	-	204	-
			404	"user not found"
			401	"invalid authorization token"
/block/:emitterId/:targetId	DELETE	-	204	-
			404	"no such block relationship was found"
			401	"invalid authorization token"

3.1.5 Requêtes - Actions incluant une dépense:

Les requêtes envoyées au serveur de l'application permettent de gérer l'argent des comptes utilisateurs de Poly Quiz.

Chemin (URL)	Méthode	Requête (corps)	Réponse	
			Code	Body
/account/:accountId/buy:itemId	POST	-	204	-
			404	"account not found"
			401	"invalid authorization token"
/account:userId/canEnterGame/:gameId	GET	-	200	boolean

3.2 Protocole WebSocket:

Les Websockets suivants permettent d'assurer une communication bidirectionnelle entre un client et le serveur ce qui permet l'affichage dynamique en temps réel. Les différents Websocket permettent à plusieurs personnes de jouer en même temps à un quiz et cela en temps réel.

Nom de l'événement	Source	Description	Destination	Données
'endQuestion'	Serveur	Avertit la fin d'une question	Client	
'getScore'	Client	Le client veut obtenir son score	Serveur	PlayerUsername
'timeTransition'	Serveur	Envoie événement de la transition entre deux questions chaque seconde	Client	TimeStamp
'finalTimeTransition'	Serveur	Dernière transition (entre dernière question et vue de résultats) envoyée à chaque seconde	Client	TimeStamp
'removedFromGame'	Serveur	Fait quitter le joueur de la vue d'attente ou de la vue du joueur	Client	
'startTransition'	Client	L'organisateur demande au serveur de commencer la transition entre deux questions	Serveur	Room
'showResult'	Client	L'organisateur demande au serveur d'envoyer les résultats finaux à chaque joueur	Serveur	Room

'nextQuestion'	Client	L'organisateur demande au serveur de passer à prochaine question	Serveur	Room
----------------	--------	--	---------	------

'refreshChoices Stats'	Serveur	Le serveur envoie les statistiques utilisées pour l'histogramme à temps réel	Client	choicesStatsValue: number[]
'getInitialQuestion'	Serveur	Le serveur envoie les données nécessaires pour la première question	Client	InitialQuestionData
'getNextQuestion'	Serveur	Le serveur envoie les données de la question demandée par l'organisateur	Client	NextQuestionData
'removedPlayer'	Serveur	Le serveur informe toute la salle qu'un joueur a quitté	Client	PlayerUsername
'gatherPlayersUsername'	Client	Un client demande au serveur la liste des joueurs afin d'avoir le classement des joueurs	Serveur	Room
'playerJoin'	Client	Lorsqu'un joueur joint la partie, avec son nom d'utilisateur	Serveur	PlayerUsername

'validateUsername'	Client	Un client demande au serveur de valider sa réponse, la réponse est retournée en callback	Serveur	userNameVerifResult
'validateRoomID '	Client	Permet de valider si on est dans le bon salon avec le roomId	Serveur	Room

'newMessage'	Client	Permet au client d'envoyer un nouveau message dans le chat	Serveur	PlayerMessage
newAudioMessage	Client	Permet au client d'envoyer un nouveau message audio dans le chat	Serveur	PlayerAudioMessage
audioMessageChunk	Client	Envoie une partie d'un fichier audio à travers le socket	Serveur	audioMessageChunk
'getMessages'	Client	Permet au client de recevoir un message émis par un autre joueur afin de l'afficher dans le chat	Serveur	Room
'getUsername'	Client	Demande l'accès à son propre nom d'utilisateur pour utiliser dans le clavardage	Serveur	Room

'messageReceived'	Serveur	Lorsqu'un joueur envoie un message, le serveur envoie cet événement avec le message à tous les utilisateurs	Client	Message
'audioMessageReceived'	Serveur	Lorsqu'un joueur envoie un message, le serveur envoie cet événement avec le message à tous les utilisateurs	Client	PlayerAudio Message
'audioMessageChunk'	Serveur	Le serveur envoie les chunks à chaque client au fur et à mesure qu'il les reçoit.	Client	audio Message Chunk
'soundboardPlayed'	client	un client informe le serveur qu'il fait jouer un son sur le soundboard	Serveur	Soundboard
'soundboardPlayed'	Serveur	le serveur informe les clients qu'un son a été joué sur le soundboard	Client	Soundboard
'stopTimer'	Client	Permet d'arrêter la minuterie	Serveur	Room
'createRoom'	Client	Permet à l'organisateur de créer un room unique	Serveur	Room

'banPlayer'	Client	Permet à l'organisateur de bannir un joueur de la partie indéfiniment	Serveur	PlayerUsername
'toggleRoomLock'	Client	Permet à l'organisateur de verrouiller le room avant de lancer la partie ou de la déverrouiller	Serveur	Room

'start'	Client	Permet à l'organisateur de lancer la partie et de notifier tous les joueurs du début de la partie	Serveur	StarTime
'gatherPlayersUsername'	Client	Permet au client de récupérer le nom des joueurs d'une partie	Serveur	Room
'newPlayer'	Serveur	Notifie les clients de l'apparition d'un nouveau joueur	Client	userInfo[]
'removedFromGame'	Serveur	Notifie le client qui a été déconnecté de la partie afin de fermer son socket et le rediriger	Client	
'removedPlayer'	Serveur	Notifie tous les clients du room du retrait d'un joueur	Client	PlayerUsername
'time'	Serveur	Permet l'envoi du temps pendant le jeu sans mode panique	Client	TimeStamp

'submitAnswer'	Client	Soumission des réponses finales du client	Serveur	PlayerAnswerData
'updateSelection'	Client	Enregistrer les sélections de réponses du client	Serveur	PlayerSelection
'pause timer'	Client	Met la minuterie en pause niveau client	Serveur	Room

'pause timer'	Serveur	permet de mettre la minuterie en pause niveau serveur	Client	
'panic mode'	Client	Quand l'organisateur active le mode panique	Serveur	Room
'panic mode'	Serveur	Le serveur broadcast à tous les clients du room le nouveau temps	Client	
'modified QRL'	Client	Permet au serveur de tenir en compte les modifications	Serveur	PlayerUsername
'updateQRldata'	Serveur	Permet de mettre à jour l'histogramme (décrémenté si vrai, incrémente si faux)	Client	boolean
'QRlending'	Serveur	Indique au joueur que l'évaluation est en cours	Client	Room

'QRLEvaluation'	Client	Quand l'organisateur note la réponse d' un joueur	Serveur	PlayerEvaluation
'Suggest Correction'	Client	le client demande au serveur qu'il contacte OpenAI pour que la question soit automatiquement corrigée	Serveur	PlayerUsername
'suggestedCorrection'	Server	le serveur renvoie la suggestion de correction à l'administrateur	Client	PlayerUsername
'AIQRLEvaluated'	Serveur	Signale au client que toutes les réponses QRL sont évaluées	Client	
'chatBanned'	Serveur	informe le client s'il peut publier dans le chat	Client	boolean
'moneyChanged'	Serveur	signale au client que sa quantité d'argent a changé	Client	
'expChanged'	Serveur	signale au client que sa quantité d'expérience a changé	Client	

4. Interfaces et Enums utilisés

Interfaces Communes:

Nom	Description	Component	
		nom	type
NextQuestion Data	Contient l'information pour passer à la prochaine question	question	QuizQuestion
		index	number
		isLast	boolean
		numberOfQuestions	number
QuizQuestion	Contient les informations nécessaires à une question d'un quiz	type	QuestionType
		text	string
		points	number
		choices	QuizChoice
Quiz	Contient l'information nécessaire pour la création d'un quiz	id	string
		title	string
		lastModification	string null
		questions	QuizQuestion[]
		visible	boolean
QuizChoice	Contient les informations nécessaires à un choix de réponse dans une question	text	string
		isCorrect	boolean
Challenge	Contient l'information nécessaire pour décrire tous les challenges. Chaque type est associé à un string identifiant, et les champs "amount" et playerId permettent de rendre un challenge spécifique à un	type	string
		amount	number null
		playerId	string null

	autre joueur (ex: soit deux fois plus rapide que joueur X) ou à spécifier une quantité (arrivé premier à 3 Quiz de suite)		
Config	Contient les informations du choix de thème d'un utilisateur	userId	string
		userName	string
		themeIndex	number
		languageID	string
		profilePicture	ImageData
Message	Contient l'information nécessaire pour envoyer un message	sender	string
		content	string
		time	string
Score	Contient les informations nécessaires pour pouvoir utiliser un score	points	number
		isBonus	Boolean
PlayerUserna me	Contient les informations nécessaires d'un joueur pour pouvoir le connecter ou le déconnecter correctement	room	Room
		username	string
PlayerMessag e	Contient les informations nécessaires pour affilier un message envoyé à un joueur	room	Room
		message	Message
PlayerAudioM essage	Contient les informations nécessaires pour décrire un message audio	room	Room
		audioMessageChunk Count	number
		audioMessageId	string
AudioMessage Chunk	Contient les informations d'un chunk et les informations permettant d'assembler ce chunks aux autres pour reconstruire le message audio	messageId	string
		chunkData	string
		chunkIndex	number

Soundboard	identifie un son joué dans une pièce	room	Room
		playerId	string
		soundId	number
PlayerTime	Contient les informations nécessaires pour obtenir le temps de réponse d'un joueur	room	Room
		time	number
PlayerAnswer Data	Contient les informations nécessaires pour pouvoir mettre à jour les réponses validées d'un joueur	room	Room
		answer	String[]
		timer	number
		username	string
PlayerSelection	Contient les informations nécessaires pour pouvoir mettre à jour les sélections de joueurs sur une question	room	Room
		isSelected	boolean
		index	number
UserInfo	Contient les informations sur un compte qui sont disponible à un utilisateur externe	username	string
		id	string
		prestigePoint	number
		profilePicture	ImageData
		userStatistics	gameStats
GameStats	Contient les statistiques historiques de parties d'un joueur.	gameNumber	number
		winsNumber	number
		averageCorrectPercentage	number
		averageCorrectNumber	number

		er	
		averageTimePerGame	number
Imagedata	Contient une image encodée en base64.	data	string
AuthorizationNotice	Contient les informations nécessaires pour gérer l'authentification	authorizationToken	String
		timeDelivered	number
		maxAge	number
Experience	Contient les informations sur un utilisateur, incluant son classement prestige et son expérience résiduelle	Prestige	number
		expérience	number
TimeStamp	Contient une donnée de temps.	timeValue	number
StartTime	contient les information sur le temps de départ d'une partie	room	Room
		time	TimeStamp
Room	contient les informations sur une salle	roomId	number
		name	string
HistogramValues	information décrivant l'histogramme	values	number[]
		labels	string[]
UserNameVerificationResult	Décrit un résultat d'évaluation d'un username. Le champ errorMessage est vide si le username est déclaré valide	isValid	boolean
		errorMessage	string null
PlayerEvaluation	Contient le résultat d'une évaluation de réponse de QRL	player	PlayerUsername
		points	number

Enum Communs:

Nom	Description	Valeur	Signification
QuestionType	Permet de différencier le type de question utilisé	0	la question est une question à choix multiple (QCM)
		1	la question est une question à réponse longue (QRL)

Interfaces client:

Nom	Description	Component	
		nom	type
GameServiceInterface	Contient les informations utiles afin d'initier l'interface de jeu	question	QuizQuestion null
		locked	boolean
		validated	boolean

Interfaces Serveur:

Nom	Description	Component	
		nom	type
Answers	Contient les informations nécessaires pour pouvoir enregistrer les informations sur les réponses choisies d'un joueur en plus de son temps	answers	string[]
		time	number